

SOI CERTIFICATE MANAGEMENT PLATFORM

Developer Architecture, API, Security, Workflow & 12-Prompt Delivery Plan

Purpose A production-oriented development specification for building a fully self-hosted, modular, multi-tenant certificate creation, issuance, verification and management platform on the same SOI CMS foundation used by MyApps. Third-party rendering/storage/queue services are intentionally optional future connectors, not dependencies.

Product	SOI Certificate Management Platform
Architecture	SOI CMS plugin; modular; multi-tenant; local-first
Primary surfaces	/super-admin, /manage, /console, /docs, /verify/{token}
Delivery model	12 cumulative Update Center ZIP releases
Deployment constraint	Web-only install/update/migration/test; no SSH or CLI requirement
Document version	1.0 - Development Baseline

Prepared for development and iterative release testing

Contents

1. Document Purpose, Scope and Non-Negotiables	19. Audit, Evidence and Compliance-Oriented Design
2. Product Surfaces and User Experience	20. Storage Management
3. High-Level Architecture	21. Security Architecture
4. Multi-Tenancy Model	22. Performance and Reliability
5. Access Control and Permission Model	23. Scheduling, Jobs and Concurrency Safety
6. Certificate Data Model and Registry	24. Reporting and Dashboard
7. Template Management and Versioning	25. Documentation Requirements (/docs)
8. Visual Certificate Designer	26. Database Schema - Proposed Entities
9. Local PDF Rendering Engine	27. Status Transition Rules
10. QR Code and Verification Management	28. Operational Error Codes
11. Certificate Creation and Issuance Pipeline	29. Testing Strategy
12. Manual, Form and Automatic Triggers	30. Update Center Packaging and Deployment
13. Scheduling Management	31. Twelve-Prompt Development Roadmap
14. API Architecture	32. Per-Prompt Mandatory Deliverables
15. API Reference - Proposed v1	33. Acceptance Checklist for 100% Planned Build Complete
16. Idempotency, Duplicate Prevention and Numbering	34. Future Extensions - Deliberately Out of v1 Core
17. Webhooks and External Integration	35. Developer Handoff Template
18. Certificate Search, Management and Operations	36. Final Architecture Decision Summary

This document is a design baseline. Implementation details may be refined during testing, but security boundaries, tenant isolation, immutable issuance history and cumulative release discipline should not be weakened without an explicit architecture decision.

1. Document Purpose, Scope and Non-Negotiables

This document is the implementation baseline for the certificate platform. It defines how the application should behave, how tenants are isolated, how certificates are designed and issued, how APIs authenticate, how verification works, how scheduled and automatic issuance is controlled, and how the product is delivered through cumulative SOI CMS Update Center ZIP packages.

1.1 Product goals

- Create certificates from reusable, versioned templates without manually editing PDFs.
- Record every issued certificate in a tenant-scoped authoritative registry.
- Embed a QR code on every issued certificate that resolves to a verification record.
- Support manual, form, API, integration, scheduled and rule-driven issuance through one central issuance service.
- Keep the initial product fully self-hosted using the existing server, PHP runtime, MySQL and local storage.
- Preserve an abstraction layer so external renderers, object storage, queues or CDN services can be added later without rewriting the certificate domain logic.
- Provide strong multi-tenant isolation, RBAC, auditable actions and safe lifecycle management.
- Remain deployable and supportable without SSH/CLI access.

1.2 Non-negotiable engineering constraints

Rule	Required behavior	Rationale
Same CMS foundation	Build as a first-class SOI CMS plugin/module set using existing authentication, sessions, CSRF, Update Center and route conventions.	Reduces new infrastructure and keeps platform behavior consistent with MyApps.
Local-first	No Cloudflare, AWS, Redis or queue is required for v1.	Volume is low; infrastructure should match actual need.
Multi-tenant by design	All tenant-owned records are scoped and authorized by tenant context.	Prevents cross-organization data leakage.
Cumulative releases	Each prompt creates one complete upgrade ZIP containing all previous work.	Every release can be installed/tested independently.
Web-only operations	Migrations, diagnostics and tests needed for deployment must be runnable from authorized web surfaces.	No SSH/CLI dependency.
Immutable issuance evidence	Issued certificate snapshots and template version references cannot silently change.	Preserves historical integrity.
Modular interfaces	Rendering, storage, notification and trigger adapters use interfaces.	Allows future third-party connectors.

1.3 Definition of “complete”

The planned build is considered functionally complete when all 12 main prompts have passed their acceptance gates, all production routes render without fatal/authorization errors for permitted users, core workflows are verified by automated and browser-level tests, a fresh install and an upgrade install both succeed through Update Center, and there are no known severity-1 or severity-2 defects. Repair prompts such as “6.1” may be inserted if testing uncovers defects; those are fixes, not additional planned product scope.

2. Product Surfaces and User Experience

2.1 Route map

Surface	Audience	Purpose	Authentication
/super-admin	Platform administrators	Platform configuration, tenant oversight, system health, global defaults, module registry and support tooling.	CMS login + platform permission
/manage	Tenant owners/admins	Tenant settings, templates, forms, API clients, issuance policies, users, branding, audit and reporting.	CMS login + active tenant membership
/console	Issuers/operators	Day-to-day certificate issuance, lookup, status actions and operational workflows.	CMS login + certificate permissions
/docs	Authorized developers/admins	Product usage, API reference, auth, webhook, integration examples and data model guidance.	Configurable; default authenticated for internal docs
/verify/{token}	Recipients/verifiers	Public or policy-protected verification of a single certificate.	Public by default, with tenant-selectable protection modes
/api/v1/...	External applications	Programmatic issuance, lookup, status and webhook management.	API client credentials/scopes

2.2 Portal separation

The portal split is a security boundary as well as a navigation choice. Platform administrators manage the SaaS/service layer; tenant administrators manage one organization; operators use the console for issuance; public verification is isolated from administrative routes. A user who can log in to the CMS is not automatically entitled to every certificate action unless the platform policy explicitly grants it.

Administrative bootstrap For initial deployment, the CMS platform administrator should be able to bootstrap certificate-platform access through a controlled one-time/explicit administrative flow. After bootstrap, normal permission checks apply. This avoids the “admin can access CMS but certificate platform says Access Denied” problem while still retaining an RBAC model for ongoing delegated access.

3. High-Level Architecture

3.1 Logical architecture

SOI CMS

```

|
+-- Authentication / Session / CSRF / Update Center
|
+-- Certificate Platform Plugin
|
+-- Platform Core / Module Registry / Schema
+-- Tenancy / Membership / RBAC
+-- Template Management / Visual Designer
+-- Certificate Issuance Service
+-- Local Rendering Adapter
+-- QR + Verification Service
+-- Forms / Triggers / Scheduling
+-- API / API Clients / Webhooks
+-- Audit / Reporting / Search
+-- Storage Adapter (Local v1)
+-- Notification Adapter (optional connectors later)

```

MySQL = authoritative metadata and transactional state

Local filesystem = generated PDFs and managed template assets in v1

3.2 Module boundaries

Module	Responsibilities	Depends on	Must not own
Core	Plugin bootstrap, schema, module registry, feature flags, health/version	SOI CMS	Tenant business rules
Tenancy	Tenant resolution, membership and status enforcement	Core, CMS users	Certificate logic
Authorization	Roles, permissions, policy checks, API scopes	Tenancy	Rendering
Templates	Templates, versions, variables, assets, publication state	Tenancy, Authorization	Issuance state
Designer	Visual editing and preview UX	Templates	Direct DB writes bypassing services
Issuance	Validation, idempotency, snapshots, lifecycle creation	Templates, Authorization	Rendering internals
Rendering	Render request to PDF	Issuance snapshot	Authorization

	using local adapter		
Verification	Tokens, public views, verification policy and events	Issuance	Template editing
Forms	Configurable input forms and mapping to issuance payload	Issuance	Separate issuance implementation
Scheduling	Scheduled/rule-driven triggers and web-safe execution	Issuance	PDF internals
API	Client auth, scopes, REST resources, errors, idempotency	Authorization, Issuance	Business rules duplicated in controllers
Audit	Immutable event capture and query	All domain services	Mutating domain records
Storage	Local file paths, integrity hashes, later connector interface	Core	Certificate status rules

3.3 Recommended source layout

```

plugins/certificates/
plugin.php
manifest.json
migrations/
src/
  Core/
  Tenancy/
  Authorization/
  Templates/
  Designer/
  Issuance/
  Rendering/
  Verification/
  Forms/
  Scheduling/
  Api/
  Webhooks/
  Audit/
  Storage/
  Reporting/
  Docs/
  Http/
views/
assets/
tests/
docs/
updates/

```

4. Multi-Tenancy Model

4.1 Tenant ownership rules

Every tenant-owned business object must carry a tenant identifier at the database boundary. Requests do not get to choose arbitrary tenant IDs. Tenant context is resolved from authorized membership/session context, then passed into repositories and services. Cross-tenant reads and writes must fail closed.

- Tenant status: active, suspended, archived. Suspended tenants cannot issue new certificates; platform administrators can still inspect/manage them.
- Membership status: active/inactive; inactive membership cannot access tenant surfaces.
- Tenant branding: legal/display name, logo, theme, verification presentation, default issuer identity.
- Tenant feature entitlements: forms, API, scheduling, bulk, private verification, webhooks, etc.
- No tenant can enumerate another tenant's template IDs, certificate IDs, API clients, form submissions or audit events.

4.2 Suggested tenant-related tables

Table	Key fields	Indexes	Notes
cert_tenants	id, slug, display_name, status, created_at	UNIQUE(slug), INDEX(status)	Can map to existing shared tenant table if CMS/MyApps tenant service is reusable.
cert_memberships	tenant_id, user_id, role_id, status	UNIQUE(tenant_id,user_id), INDEX(user_id,status)	Membership-level access.
cert_roles	tenant_id/null, name, scope, is_system	INDEX(tenant_id)	System and tenant-defined roles.
cert_role_permissions	role_id, permission_key	UNIQUE(role_id,permission_key)	Normalized permission mapping.
cert_settings	tenant_id, setting_key, value_json	UNIQUE(tenant_id,setting_key)	Tenant-specific preferences.

5. Access Control and Permission Model

5.1 Human roles

Roles should be understandable to non-technical tenant administrators. Permissions remain granular internally so roles can be customized later.

Role	Templates	Issue	Manage certs	Integrations	Admin
Platform Super Admin	All tenants	All tenants	All tenants	All tenants	Platform
Tenant Owner	Manage	Yes	Yes	Manage	Tenant
Tenant Admin	Manage	Yes	Yes	Manage if granted	Tenant
Template Designer	Manage	Optional	View	No	No
Issuer	View published	Yes	Own/allowed	No	No
Auditor/Viewer	View	No	View	View metadata	No

5.2 Permission keys

```
platform.read
platform.manage
tenants.read
tenants.manage
roles.manage
templates.read
templates.create
templates.update
templates.publish
templates.archive
certificates.read
certificates.issue
certificates.download
certificates.revoke
certificates.replace
certificates.cancel
forms.manage
forms.submit
schedules.manage
api_clients.manage
webhooks.manage
audit.read
reports.export
verification.manage
```

5.3 Authorization rules

1. Authenticate the actor (CMS session or API client).

2. Resolve tenant context from an authorized membership or API credential.
3. Check tenant status and membership/client status.
4. Check the required permission or API scope.
5. Load the target object with tenant_id in the query predicate.
6. Apply object-level rules (for example, issuer may only revoke certificates they are permitted to manage).
7. Write an audit event for sensitive operations and denied policy events when useful.

6. Certificate Data Model and Registry

6.1 Certificate identity

Each issued certificate has two distinct identifiers: a human-friendly certificate number and a non-guessable verification token. The certificate number may follow a tenant-configurable format; the verification token should be random and never sequential.

```
certificate_number: SOI-2026-000184
verification_token: 128+ bits of cryptographically secure randomness
verification_url: /verify/{verification_token}
```

6.2 Core certificate fields

Field	Purpose	Mutable after issue?	Security/behavior
tenant_id	Owner tenant	No	Always scoped/indexed
certificate_number	Human reference	Normally no	Unique within defined tenant namespace
verification_token_hash / token	Verification lookup	No	Prefer storing hashed token when implementation allows
template_id + template_version_id	Design source	No	Pin exact version
recipient snapshot	Name and public recipient fields	No except controlled correction workflow	Do not silently overwrite history
payload_json	Resolved custom variables	No	Snapshot after validation
status	Lifecycle state	Yes via audited transition	Finite state machine
issued_at	Issuance timestamp	No	Server-side authoritative time
expires_at	Optional expiry	Controlled	Tenant policy
file_path/object_ref	Rendered artifact location	May move through storage adapter	Do not expose raw internal path
file_sha256	Integrity hash	Recomputed only on controlled replacement	Detect unexpected file changes
created_by_type/id	Actor attribution	No	Human/API/schedule/form
source_type/source_id	Trigger attribution	No	Manual, form, API, schedule, webhook, import

6.3 Lifecycle state machine

DRAFT (optional pre-issue record)

```

|
+--> ISSUING --> ISSUED
      | | |
      | | +--> EXPIRED (time/policy)
      | +-----> REVOKED (explicit invalidation)
      +-----> REPLACED (superseded by another certificate)

```

CANCELLED = issuance intentionally abandoned before valid issue.

A certificate should never be hard-deleted from the normal product interface after it has reached ISSUED. Revocation and replacement preserve evidence and history.

7. Template Management and Versioning

7.1 Template states

- Draft - editable working version.
- Published - eligible for issuance.
- Archived - retained for history but not selectable for new issuance.
- Superseded - older published version retained because certificates reference it.

7.2 Template content

A template should be represented as structured JSON plus managed assets rather than a single uncontrolled HTML blob. Structured elements make validation, editing, previews and renderer portability much safer.

```
{
  "page": {"size":"A4", "orientation":"landscape"},
  "background_asset_id": 241,
  "elements": [
    {"type":"text", "x":120, "y":88, "w":610, "h":70, "value":"Certificate of Excellence"},
    {"type":"variable", "key":"recipient_name", "x":150, "y":250, "w":550, "h":55},
    {"type":"image", "asset_id":812, "role":"signature"},
    {"type":"qr", "value":"verification_url", "x":690, "y":420, "size":95}
  ]
}
```

7.3 Variable definitions

Variable	Type	Required	Example	Visibility
recipient_name	string	yes	Arunima Rai	certificate + verification
certificate_title	string	yes	Leadership Excellence	certificate + verification
issue_date	date	auto/default	2026-09-27	certificate + verification
certificate_number	system	auto	SOI-2026-0012	certificate + verification
verification_url	system	auto	https://.../verify/...	QR only
department	string	optional	Operations	tenant policy
custom:*	typed custom field	configurable	score=97	tenant policy

7.4 Publishing rule

Critical integrity rule Editing a published template creates a new draft version. Publishing that draft creates a new immutable version. Existing certificates continue to point to the exact version used when they were issued.

8. Visual Certificate Designer

8.1 Designer capabilities

- Canvas with accurate page size and orientation (A4, Letter, custom dimensions).
- Drag, resize, align, snap-to-grid and z-order controls.
- Text blocks with font, size, weight, color, alignment and line-height.
- Dynamic variables with sample-data preview.
- Images: logos, seals, badges, signatures and background artwork.
- Lines, rectangles and simple decorative shapes.
- QR code element bound to the system verification URL.
- Guides/safe areas to avoid print clipping.
- Undo/redo for current edit session where practical.
- Desktop-first editor with responsive management UI; the actual certificate canvas uses fixed physical proportions.
- Test-render/preview before publishing.

8.2 Asset rules

- Validate MIME type and file size server-side; never trust extension alone.
- Reject executable/vector content that cannot be safely sanitized for the chosen renderer.
- Store tenant ownership and asset purpose metadata.
- Do not permit one tenant to reference another tenant's asset ID.
- Signature assets require a stricter permission such as `templates.manage_signatures` if desired.

9. Local PDF Rendering Engine

9.1 Renderer abstraction

```
interface CertificateRendererInterface {
    RenderResult render(RenderRequest request);
}
```

LocalCertificateRenderer implements CertificateRendererInterface

Future (optional):

CloudflareBrowserRenderer implements CertificateRendererInterface

AwsLambdaRenderer implements CertificateRendererInterface

The issuance service only knows the renderer interface. It should not contain vendor-specific calls. For v1, the local renderer uses tools/libraries available on the existing host and produces deterministic PDF output from the normalized template representation.

9.2 Render request

- Exact template version snapshot.
- Validated certificate payload.
- System variables: number, issue date, verification URL/token representation.
- Resolved asset references.
- Output page dimensions/orientation.
- Render correlation ID for troubleshooting.

9.3 Render result

- PDF bytes or temporary local path.
- Page count and output size.
- SHA-256 hash.
- Render duration.
- Warnings/errors without leaking server internals to end users.

9.4 Failure behavior

An issuance record must not be marked ISSUED until rendering and final storage succeed. If rendering fails, preserve a failed/issuance-attempt record with a sanitized error code and allow authorized retry. A retry must not create duplicate certificate numbers unless the previous attempt never committed an issuance identity.

10. QR Code and Verification Management

10.1 QR behavior

Every issued certificate includes a QR code generated locally. The QR stores only the verification URL or opaque token reference - never confidential certificate payload. The verification page reads the authoritative database record.

10.2 Verification modes

Mode	Use case	Access	Displayed data
Public	General awards/training certificates	No login	Tenant-approved public fields
Masked public	Privacy-sensitive recipient names	No login	Masked recipient + essential certificate status
PIN protected	Recipient shares a secondary code	Token + PIN	Configured fields
Authenticated	Internal certificates	CMS/login or tenant SSO future	Full configured verification view
Disabled public view	Highly restricted records	Admin/authorized lookup only	No public details

10.3 Verification result

- Valid / Revoked / Expired / Replaced / Not Found / Verification Disabled.
- Certificate number and title.
- Issuing organization.
- Recipient name or masked value based on policy.
- Issue and optional expiration date.
- Optional downloadable certificate link if tenant policy permits.
- Replacement reference when certificate was superseded.
- No display of internal user IDs, source payloads, filesystem paths, API client details or audit internals.

10.4 Verification abuse controls

Verification tokens must be high-entropy so the primary defense is non-enumerability. Add lightweight request throttling, generic not-found responses, no sequential IDs in the URL, and an audit/analytics event that does not become a privacy-sensitive tracking profile. CAPTCHA is not required by default at the expected scale.

11. Certificate Creation and Issuance Pipeline

11.1 Single issuance service

```
Manual UI -----+
Form submission ----+
API request -----+--> CertificateIssuanceService --> Renderer --> Storage --> ISSUED
Scheduled job -----+
Automation rule -----+
Bulk/import row -----+
```

All sources use the same validation, authorization, idempotency, numbering, snapshot and lifecycle code. Source-specific controllers only translate input into a normalized issuance command.

11.2 Issuance steps

1. Resolve actor, tenant and source.
2. Authorize certificate.issue for the relevant source/template.
3. Load a published template version.
4. Validate required variables and type constraints.
5. Apply server-generated system variables.
6. Check idempotency/deduplication policy.
7. Reserve certificate identity/number transactionally.
8. Create immutable issuance snapshot and source attribution.
9. Create verification token and QR payload.
10. Render PDF locally.
11. Persist artifact and integrity hash.
12. Commit final ISSUED state and audit event.
13. Invoke optional post-issue hooks/webhooks/notification adapters.
14. Return certificate metadata and permitted download/verification links.

12. Manual, Form and Automatic Triggers

12.1 Manual trigger

An authorized issuer selects a published template, enters recipient/custom values, previews the resolved certificate and issues it. The UI should support saving draft input without reserving a final certificate number if that workflow is useful.

12.2 Dynamic form trigger

Tenant admins can build forms mapped to template variables. A form is not allowed to directly write certificate records; it submits a normalized issuance command after validation and policy evaluation.

Form feature	Behavior	Example	Security
Field types	text, email, number, date, select, textarea, checkbox, hidden/system	recipient_name	Server validation
Mapping	Map form field -> template variable	full_name -> recipient_name	Only variables declared by template
Approval	Optional approval before issue	Manager approves award	Permission + audit
Availability	Internal, link-only, public	Nomination form	Tenant policy
Duplicate rule	Reject/allow/replace based on configured key	employee_id + course_id	Server-side
Confirmation	Show status/reference, not sensitive internals	Request accepted	Generic errors for public forms

12.3 Automatic trigger

Automatic triggers convert trusted application events or internal rules into issuance commands. They must be explicitly configured by a tenant admin and should not be implemented as arbitrary PHP/code execution.

- Event trigger: an internal application sends an authenticated event such as course.completed.
- Rule mapping: event fields map to certificate variables.
- Conditions: issue only when configured predicates are true.
- Deduplication key: prevents the same event from issuing multiple certificates.
- Optional approval: event creates a pending request that an issuer approves.
- Audit: record rule ID, event reference, source system and result.

13. Scheduling Management

13.1 What scheduling means in this platform

Scheduling is for planned issuance or planned lifecycle actions, not for running a high-throughput queue. Examples include issuing certificates at the end of a training cohort, releasing them at a specific date/time, expiring time-limited certifications, or executing a recurring issuance rule.

13.2 Schedule types

- One-time issue at a specified tenant timezone date/time.
- One-time batch/form release.
- Recurring schedule with explicit frequency and end condition.
- Expiration evaluator for certificates with `expires_at`.
- Rule-driven schedule that queries a bounded source list and issues eligible records.

13.3 Web-only execution strategy

Because the deployment cannot depend on CLI/cron access, the product must support a secure web scheduler runner endpoint protected by a long random runner secret and/or authenticated internal scheduler call. It should also support opportunistic due-job execution during normal authenticated traffic as a fallback. The scheduler must use a database lock/lease so the same due job cannot run twice concurrently.

Important If the host already offers a web-based cron/scheduler UI, it may call the protected scheduler endpoint. The product must still remain correct if two runner calls overlap.

14. API Architecture

14.1 API principles

- Versioned under /api/v1.
- JSON request/response for control operations; binary/stream response only for explicit downloads.
- API business logic delegates to the same domain services used by the UI.
- Tenant is derived from the API client; callers cannot select arbitrary tenant IDs.
- Every write endpoint supports a request correlation ID; issuance endpoints support idempotency.
- Consistent error envelope and HTTP status use.
- Rate limits can be lightweight and local for v1, configurable per client/tenant.

14.2 API authentication

Use tenant-scoped API clients, each with its own client identifier, secret and scopes. Store API secrets only as strong one-way hashes after initial display. Never store or redisplay the raw secret after creation.

```
Authorization: Bearer <api_secret>
X-Request-ID: optional caller correlation id
Idempotency-Key: required/recommended for POST /certificates
```

For additional hardening, the product can later support HMAC request signing. For v1, a high-entropy bearer secret over HTTPS plus strict scopes and tenant binding is sufficient for the expected internal integration model.

14.3 API client lifecycle

- Create client: returns secret once.
- Assign scopes and optional template restrictions.
- Activate/deactivate without deleting historical attribution.
- Rotate secret: new secret replaces previous credential with an optional short grace period.
- Last-used timestamp and source IP metadata for operations.
- Revoke immediately on suspected compromise.

15. API Reference - Proposed v1

15.1 Certificate resources

Method	Endpoint	Scope	Purpose	Idempotent?
POST	/api/v1/certificates	certificates.issue	Issue a certificate from a published template	Yes via Idempotency-Key
GET	/api/v1/certificates/{id}	certificates.read	Read certificate metadata	N/A
GET	/api/v1/certificates/{id}/download	certificates.download	Download PDF if policy permits	N/A
POST	/api/v1/certificates/{id}/revoke	certificates.revoke	Revoke an issued certificate	Safe repeat returns current state
POST	/api/v1/certificates/{id}/replace	certificates.replace	Issue replacement and link old/new	Yes via Idempotency-Key
GET	/api/v1/certificates	certificates.read	Search/list with pagination and filters	N/A
POST	/api/v1/certificates/verify	verification.read	Server-to-server verification by token/number	N/A

15.2 Template and form APIs

Method	Endpoint	Scope	Purpose
GET	/api/v1/templates	templates.read	List published/available templates for the client.
GET	/api/v1/templates/{id}	templates.read	Retrieve public integration schema/variables.
POST	/api/v1/forms/{form_key}/submit	forms.submit	Submit to an API-enabled issuance form.
GET	/api/v1/jobs/{id}	certificates.read	Read async/scheduled job status where used.

15.3 Example issue request

```
POST /api/v1/certificates
Authorization: Bearer sk_live_...
```

Idempotency-Key: employee-174-training-2026
Content-Type: application/json

```
{
  "template_key": "leadership-excellence",
  "recipient": {
    "name": "Arunima Rai",
    "email": "arunima@example.com"
  },
  "variables": {
    "program_name": "Senior Leadership Program",
    "department": "Operations"
  },
  "issue_date": "2026-09-27"
}
```

15.4 Example response

```
HTTP 201 Created
{
  "data": {
    "id": "crt_01J...",
    "certificate_number": "SOI-2026-00184",
    "status": "issued",
    "issued_at": "2026-09-27T16:44:19+05:30",
    "verification_url": "https://cert.example/verify/opaque-token",
    "download_url": "/api/v1/certificates/crt_01J.../download"
  },
  "request_id": "req_..."
}
```

15.5 Error envelope

```
{
  "error": {
    "code": "TEMPLATE_NOT_PUBLISHED",
    "message": "The requested template is not available for issuance.",
    "field_errors": {"variables.score": ["Must be between 0 and 100"]}
  },
  "request_id": "req_..."
}
```

Do not expose SQL errors, stack traces, local filesystem paths, raw exception messages or internal class names in API responses.

16. Idempotency, Duplicate Prevention and Numbering

16.1 Idempotency

API and automation issuance must support idempotency. A tenant-scoped idempotency key maps to a request fingerprint and prior result. Repeating the same key with the same request returns the original issuance result. Reusing the key with a materially different request returns a conflict.

16.2 Duplicate policy

- Per-template optional duplicate key definition, e.g., employee_id + course_id.
- Allow, reject, require approval, or replace existing certificate according to policy.
- Duplicate decisions are made before final number reservation where possible.
- Never rely only on UI warnings; enforce at service/database boundary.

16.3 Number generation

Certificate numbers should be generated transactionally using a tenant/template numbering rule with a dedicated sequence record. Do not compute “MAX(number)+1”. Number formatting may include tenant code, year and padded sequence; the underlying sequence remains numeric and locked safely during allocation.

17. Webhooks and External Integration

17.1 Outbound webhook events

```
certificate.issued  
certificate.revoked  
certificate.replaced  
certificate.expired  
issuance.failed
```

17.2 Webhook security

- Each endpoint gets a signing secret.
- Sign raw payload using HMAC-SHA256 with timestamp header.
- Consumer verifies timestamp tolerance and signature before processing.
- Delivery log stores status code, attempt count, next retry time and sanitized response summary.
- Retries use bounded exponential backoff; repeated failure moves delivery to failed/dead state visible to tenant admin.
- No silent “done” state unless an HTTP request was actually attempted and accepted according to configured success codes.

17.3 Integration examples

- MyApps/internal HR application calls the API after a workflow reaches an approved state.
- Training system sends course.completed with a stable event ID; automation mapping issues the appropriate certificate.
- An internal portal embeds a protected form that creates a pending issuance request.
- Future mailer/WhatsApp connectors can receive certificate.issued and deliver the download/verification link.

18. Certificate Search, Management and Operations

18.1 Registry UI

- Search by certificate number, recipient, template, status and date range.
- Filter by source: manual, form, API, schedule, automation, import.
- Open detail page showing current status, immutable issuance snapshot, render metadata and lifecycle history.
- Download PDF if permitted.
- Revoke with mandatory reason.
- Replace/reissue with explicit linkage and reason.
- Export filtered metadata subject to permissions.
- Never expose a destructive delete button for issued certificates.

18.2 Certificate detail view

The detail view should clearly separate the authoritative issued data from current tenant/template configuration. It must show “Issued using Template X, Version Y” so operators understand why an older certificate may look different from today’s template.

19. Audit, Evidence and Compliance-Oriented Design

19.1 Audit events

- Tenant created/suspended/reactivated.
- Role/permission changed.
- Template created/edited/published/archived.
- API client created/rotated/revoked.
- Form or schedule created/changed.
- Certificate issued/revoked/replaced/cancelled.
- Verification policy changed.
- Sensitive asset/signature changed.
- Webhook endpoint changed.
- Production migration run and result.

19.2 Audit record fields

```
id, tenant_id, actor_type, actor_id, event_key, target_type, target_id,  
request_id, source_ip (policy-controlled), metadata_json, created_at
```

Audit records should be append-only through application code. Sensitive values such as raw API secrets, verification PINs, full webhook secrets or session tokens must never be written to audit metadata.

20. Storage Management

20.1 Local storage v1

Generated certificates and managed assets are stored locally in a non-public application-controlled directory. Downloads are served through authorization-aware controllers rather than exposing the storage directory directly.

```
storage/certificates/{tenant-key}/{yyyy}/{mm}/{certificate-internal-id}.pdf
storage/certificate-assets/{tenant-key}/{asset-id}/...
```

20.2 Integrity and retention

- Persist SHA-256 for issued PDFs.
- Backup storage together with database so metadata and files remain consistent.
- Do not overwrite issued PDF files in place during ordinary operations.
- Render replacement/corrected certificates to a new artifact and preserve linkage.
- Provide storage health checks: writable, free space threshold, missing file detection.

20.3 Future connector interface

A StorageAdapter interface allows local storage to be replaced or supplemented by R2, S3 or another object store later. The database stores an abstract object reference, not vendor-specific semantics in certificate domain code.

21. Security Architecture

21.1 Web security baseline

- HTTPS required in production.
- CMS session protections and CSRF for all state-changing browser actions.
- Output escaping in management and public verification views.
- Strict server-side validation for template variables and form submissions.
- Prepared statements/repository parameterization; never interpolate request values into SQL.
- Content Security Policy and secure response headers compatible with the designer.
- Safe upload handling and image validation.
- No direct public access to stored PDFs unless explicitly designed as signed/public links.
- Secrets encrypted where reversible storage is necessary; API secrets hashed where only comparison is required.
- Fail closed on missing tenant context, role or permission.

21.2 Tenant isolation tests

- Changing numeric IDs in URLs must not access another tenant.
- API client for Tenant A cannot fetch Tenant B objects even with a guessed ID.
- Form key/token cannot be reused across tenants unless intentionally global.
- Asset IDs, template IDs and certificate IDs are all tenant constrained at query time.
- Super-admin bypass is explicit in authorization code and audited; it is not achieved by omitting tenant filters accidentally.

21.3 Verification privacy

The tenant controls which fields are public. Recipient email, employee ID, internal comments, API source data and audit metadata are private by default. Public verification pages should include noindex metadata if the organization does not want search engines indexing certificate records.

22. Performance and Reliability

22.1 Expected workload

The stated initial load is only a few certificates per day, so the platform should optimize for correctness and low operational complexity rather than distributed infrastructure. MySQL and local rendering are sufficient.

22.2 Required indexes

```
certificates: (tenant_id, certificate_number), (tenant_id, status), (tenant_id, issued_at)
verification: unique(token_hash) or unique(token)
templates: (tenant_id, status), unique(tenant_id, template_key)
memberships: unique(tenant_id, user_id), (user_id, status)
api_clients: (tenant_id, status), unique(client_id)
audit: (tenant_id, created_at), (target_type, target_id)
schedules: (status, next_run_at)
```

22.3 Reliability behaviors

- Database transactions around number allocation and issuance state changes.
- Idempotency for external and automatic issuance.
- Render/storage failure never results in a falsely “issued” record.
- Health checks for schema version, storage writability and renderer availability.
- Retry controls are explicit and audited.
- No dependence on remote third-party services in v1.

23. Scheduling, Jobs and Concurrency Safety

23.1 Database-backed job state

Scheduled work can use a lightweight jobs table rather than Redis/queue infrastructure. Each job has status, run_after, lease_owner/token, lease_until, attempt_count and last_error_code. A runner atomically leases a due job before executing it.

23.2 Suggested states

```
PENDING -> RUNNING -> SUCCEEDED
|
+ -> RETRY_WAIT -> RUNNING
+ -> FAILED
```

23.3 Concurrency rule

Two web requests may attempt to run due jobs at the same time. The job acquisition query must atomically update/lease one eligible row and only the lease holder may complete it. This also protects scheduled issuance from duplicate certificates.

24. Reporting and Dashboard

24.1 Tenant dashboard

- Certificates issued today/this month.
- Active/revoked/expired counts.
- Most-used templates.
- Recent issuance failures requiring attention.
- Upcoming scheduled jobs.
- API client activity and webhook failures.
- Storage/renderer health summary for authorized admins.

24.2 Platform dashboard

- Tenant count and status.
- Issuance volume by tenant without exposing private payload data unnecessarily.
- System-wide renderer/storage/schema health.
- Migration/update status.
- Failed integrations and operational alerts.

25. Documentation Requirements (/docs)

25.1 Developer documentation

- Getting started and tenant/API client setup.
- Authentication and secret rotation.
- API scopes and permission mapping.
- Complete endpoint reference with request/response examples.
- Idempotency semantics.
- Webhook signatures and verification example.
- Error codes and retry guidance.
- Template integration schema and variable discovery.
- Form and automation integration patterns.
- Versioning/backward compatibility policy.

25.2 Administrator documentation

- Creating templates and publishing versions.
- Issuing/revoking/replacing certificates.
- Creating forms and approvals.
- Creating API clients.
- Configuring verification privacy.
- Configuring schedules/automatic triggers.
- Reviewing audit logs and failed jobs.
- Backup/recovery expectations.

26. Database Schema - Proposed Entities

Entity/table	Purpose	Primary relationships	Retention
cert_tenants	Tenant profile if not reusing shared tenancy	memberships, templates, certificates	Until archive policy
cert_memberships	User membership/role	tenant + CMS user	Historical changes via audit
cert_roles / role_permissions	RBAC definitions	tenant/system	Persistent
cert_templates	Logical template	tenant, versions	Persistent/archive
cert_template_versions	Immutable published/draft versions	template, assets	Never delete when referenced
cert_template_assets	Logos/backgrounds/signatures	tenant, versions	Retain while referenced
certificates	Authoritative registry	tenant, template version	Never hard-delete after issue
cert_certificate_events	Lifecycle/event history	certificate	Persistent
cert_forms	Dynamic forms	tenant, template	Archive
cert_form_submissions	Submission/request record	form, certificate optional	Policy-based retention
cert_api_clients	External credentials metadata	tenant	Retain attribution after revoke
cert_idempotency	Deduplication keys/results	tenant, API client/source	Time-bounded retention
cert_webhooks	Endpoint config	tenant	Archive
cert_webhook_deliveries	Delivery attempts	webhook, event	Operational retention
cert_schedules	Schedule definitions	tenant, template/rule	Archive
cert_jobs	Due/running job state	schedule/source	Operational retention
cert_audit_log	Security/domain audit trail	tenant, actor, target	Long retention

27. Status Transition Rules

From	Action	To	Permission	Notes
Issuing	render/store success	Issued	system	Audit issuance
Issued	revoke	Revoked	certificates.revoke	Reason required
Issued	replace	Replaced	certificates.replace	New certificate issued + linked
Issued	time/policy expiry	Expired	system	No loss of record
Draft/Pending	cancel	Cancelled	certificates.issue or manage	No valid certificate exists
Revoked	restore	Not allowed by default	-	Prefer new/replacement certificate
Replaced	reactivate	Not allowed	-	Preserve chain integrity

28. Operational Error Codes

Use stable error codes so the UI and API can respond consistently even if human-readable messages change.

```
AUTH_REQUIRED  
ACCESS_DENIED  
TENANT_SUSPENDED  
MEMBERSHIP_INACTIVE  
TEMPLATE_NOT_FOUND  
TEMPLATE_NOT_PUBLISHED  
VARIABLE_VALIDATION_FAILED  
DUPLICATE_CERTIFICATE  
IDEMPOTENCY_CONFLICT  
NUMBER_ALLOCATION_FAILED  
RENDER_FAILED  
STORAGE_FAILED  
CERTIFICATE_NOT_FOUND  
INVALID_STATUS_TRANSITION  
VERIFICATION_DISABLED  
API_CLIENT_REVOKED  
API_SCOPE_REQUIRED  
WEBHOOK_DELIVERY_FAILED  
SCHEDULE_LOCKED
```

29. Testing Strategy

29.1 Test layers

Layer	Examples	When run	Gate
Unit/domain	numbering, transitions, validation, scope checks	Every prompt	Must pass
Repository/schema	tenant predicates, migrations, indexes, uniqueness	Schema-changing prompts	Must pass
HTTP/controller	auth, CSRF, routes, API status codes	Every prompt	Must pass
Browser smoke	/super-admin, /manage, /console, /docs, designer, verify	UI prompts and final	Must pass
Security	cross-tenant tampering, scope escalation, upload safety	RBAC/API/final	Must pass
Rendering	known template -> valid PDF, QR present, assets resolved	Renderer onward	Must pass
Upgrade	previous ZIP -> current ZIP	Every cumulative release	Must pass
Fresh install	empty supported baseline -> current ZIP	Every major milestone/final	Must pass

29.2 Must-have end-to-end scenarios

1. Create tenant, assign owner, log in to /manage.
2. Create template, add variables/assets/QR, publish version 1.
3. Issue manually; PDF generated and QR verifies as valid.
4. Edit template and publish version 2; old certificate remains tied to v1.
5. Revoke certificate; public verification immediately reports revoked.
6. Replace certificate; old verification reports replaced and links appropriately.
7. Create API client with issue scope; issue using Idempotency-Key; replay returns same certificate.
8. Attempt cross-tenant API lookup; receive not found/forbidden without leakage.
9. Create form mapped to template; submit and issue/approve according to policy.
10. Create scheduled issuance; overlapping runners execute it once.
11. Upgrade from previous cumulative ZIP without losing data or activation.

30. Update Center Packaging and Deployment

30.1 Package rules

- Every prompt produces one cumulative Update Center-compatible ZIP.
- manifest.json includes product version, schema version, compatibility and migration metadata expected by SOI CMS.
- Migrations are idempotent or safely version-gated.
- Plugin activation state must be preserved through upgrade where the CMS supports it.
- No hardcoded production domain.
- No manual SQL required for normal install/upgrade; privileged web migration runner can recover/inspect migration state.
- Every release includes docs/development-status.md and release notes.

30.2 Versioning example

```
certificates-1.0.0-prompt1.zip  
certificates-1.1.0-prompt2.zip  
certificates-1.2.0-prompt3.zip  
...  
certificates-1.11.0-prompt11.zip  
certificates-2.0.0-production.zip
```

31. Twelve-Prompt Development Roadmap

#	Prompt	Build focus	Exit gate
1	Prompt 1 - Platform Foundation	Plugin bootstrap, module registry, migrations, schema/version health, base route shells, diagnostics and Update Center package.	Fresh install succeeds; plugin activates; health/version/migration surfaces work; no dead routes presented as finished.
2	Prompt 2 - Multi-Tenancy & RBAC	Tenant context, memberships, roles/permissions, bootstrap access, /super-admin, /manage and /console authorization.	Platform admin can access required admin surfaces; tenant access is isolated; cross-tenant tests pass.
3	Prompt 3 - Certificate Templates & Versioning	Template model, variables, assets, states, versions and publish/archive workflows.	Create/publish template; version history is immutable and tenant-scoped.
4	Prompt 4 - Visual Certificate Designer	Canvas editor, text/variables/images/signatures/shapes/QR placeholder, preview and saved structured definition.	A tenant can visually design and preview a complete certificate template.
5	Prompt 5 - Local PDF Rendering Engine	Renderer interface, local renderer, asset/font handling, PDF integrity hashing, render diagnostics and failure states.	Known template renders to valid PDF locally with deterministic dimensions.
6	Prompt 6 - Certificate Issuance, QR & Public Verification	Central issuance service, numbering, snapshots, QR generation, registry, download and /verify/{token}.	Manual end-to-end issue -> PDF -> QR -> authoritative verification succeeds.
7	Prompt 7 - Certificate Lifecycle & Audit History	Revoke, expire, cancel, replace/reissue, lifecycle events, certificate detail history and audit coverage.	All allowed transitions work and invalid transitions fail safely.
8	Prompt 8 - Manual Issuance & Dynamic Forms	Issuer UX, configurable forms, variable mapping, approval options,	Form/manual triggers call the same issuance service and produce correct

		duplicate policy and public/internal form modes.	audit/source attribution.
9	Prompt 9 - API, API Keys, Scopes & Webhooks	Tenant API clients, bearer auth, secret rotation, scopes, REST v1, idempotency, request IDs and signed outbound webhooks.	External app can securely issue/read/revoke within granted scope; webhook signatures and retry logs work.
10	Prompt 10 - Bulk Certificate Issuance	CSV/Excel/paste import, mapping, server-side chunking, progress, skip/fail details and retry/export.	Large input is handled without arbitrary small hard limit or request timeout pattern.
11	Prompt 11 - Dashboard, Branding, Search, Export & Documentation	Operational dashboards, registry search/filter/export, tenant branding/settings, /docs, API examples, health/support UX.	Feature-complete administrator/operator experience with usable documentation.
12	Prompt 12 - Production Hardening, Security, Performance & Final Release	Full regression, browser tests, tenant isolation attack tests, migration/upgrade tests, storage/renderer recovery, performance indexes, accessibility/UX fixes and final cumulative ZIP.	Production acceptance checklist passes; fresh install and upgrade are green; no known critical/high defects.

32. Per-Prompt Mandatory Deliverables

Every prompt must instruct the coding AI to leave the repository in a testable, handoff-ready state. The following items are mandatory in every build prompt:

1. Implement only the scoped prompt plus any necessary repair of previously discovered defects.
2. Preserve all completed functionality from prior prompts.
3. Increment application/schema versions appropriately.
4. Add or update migrations; do not require manual database edits for ordinary upgrades.
5. Add automated tests for the new behavior and regressions for bugs fixed.
6. Run the full available test suite, not only newly added tests.
7. Run HTTP/browser smoke checks for affected routes.
8. Create one cumulative Update Center ZIP and validate its manifest/layout.
9. Update docs/development-status.md with completed, partial, pending, known defects, tests, versions, routes and next prompt.
10. If a serious defect remains, stop and report it rather than labeling the phase complete.

33. Acceptance Checklist for “100% Planned Build Complete”

- ☐ All 12 planned prompts completed and their exit gates passed.
- ☐ Latest ZIP installs cleanly on a fresh supported SOI CMS installation.
- ☐ Latest ZIP upgrades cleanly from the previous production/release candidate without losing data or activation.
- ☐ /super-admin, /manage, /console and /docs render for authorized users without 404/fatal errors.
- ☐ Public/policy-protected /verify/{token} behaves according to verification settings.
- ☐ Template versioning prevents historical certificate drift.
- ☐ Local PDF rendering, QR creation and local storage operate without third-party services.
- ☐ Manual, form, API, schedule/automation and bulk triggers all converge on the central issuance service.
- ☐ API credentials/scopes, idempotency and webhook signatures are tested.
- ☐ Revoke/replace/expire behavior is reflected immediately in verification.
- ☐ Cross-tenant access tests pass for UI, API, templates, assets, certificates and forms.
- ☐ All state-changing browser routes have CSRF protection; API routes use client auth/scopes.
- ☐ Audit coverage exists for security-sensitive and certificate lifecycle actions.
- ☐ Storage/renderer/schema health can be checked from authorized web UI.
- ☐ No mandatory Redis, queue, Cloudflare, AWS, SSH or CLI dependency exists.
- ☐ Developer/admin/API documentation reflects actual implemented behavior.
- ☐ No known critical/high defects remain.

34. Future Extensions - Deliberately Out of v1 Core

These are supported by the architecture but should not be allowed to complicate the initial self-hosted release:

- Cloudflare/AWS remote rendering connectors.
- R2/S3/object-storage adapter.
- External queue/worker connector for very high issuance volume.
- Email, SMS or WhatsApp delivery connectors to the wider SOI service ecosystem.
- Enterprise SSO/SAML/OIDC for verification or administrator access.
- Cryptographic document signing / qualified digital signatures where legally required.
- Blockchain or public ledger anchoring (not required for ordinary certificate verification).
- Advanced analytics, billing/usage metering and external tenant self-service plans.
- Certificate wallets/badges/open credentials interoperability.

35. Developer Handoff Template

Each build phase should update the following handoff structure so another AI/developer can continue without reconstructing the project history:

```
# Development Status
Product version:
Schema version:
Current cumulative ZIP:

## Completed in this phase
## Fixed in this phase
## Partially completed
## Not started / next prompt
## Known defects and severity
## Routes added/changed
## Database migrations
## Authorization changes
## Test suites and counts
## Browser/HTTP smoke results
## Packaging validation
## Deployment/activation notes
## Architectural decisions that must be preserved
```

36. Final Architecture Decision Summary

Decision	Baseline
Platform	Existing SOI CMS, implemented as modular certificate plugin/modules.
Tenancy	Multi-tenant from the database/service layer; shared conventions with MyApps.
Rendering	Local/self-hosted v1 behind a renderer interface.
Storage	Local/self-hosted v1 behind a storage interface.
Certificate truth	MySQL registry + immutable issuance snapshot + artifact hash.
Verification	Opaque QR token -> authoritative verification page; configurable privacy/auth.
Triggers	Manual, form, API, automation, schedule and bulk; all use one issuance service.
API auth	Tenant-scoped high-entropy bearer secret, hashed at rest, scopes, rotation and idempotency.
RBAC	Platform admin + tenant owner/admin/designer/issuer/viewer with granular permissions.
Scheduling	Database-backed jobs with web scheduler runner and locking; no mandatory queue.
Deployment	12 cumulative Update Center ZIP releases; web-only migration/testing support.
Completion gate	Final full regression/security/upgrade/fresh-install acceptance with no known critical/high defects.

Development baseline This plan is intentionally designed so the first production version can run entirely on the existing host. External cloud services are future adapters, not architectural requirements. That keeps the initial platform reliable, understandable, inexpensive and consistent with the MyApps/SOI CMS development model.